

Verilog HDL Projects

Thejaswini K | ECE Graduate | Maven Silicon Trained

1. FSM Sequence Detector (Moore Machine)

Detects the binary sequence **1011** on a serial input using a Moore FSM. Includes state encoding, sequential state register, combinational next-state logic, and output logic. Testbench validates all state transitions.

```
// FSM Sequence Detector (Moore Machine) - Detects "1011"
// Author: Thejaswini K | Maven Silicon Training

module fsm_sequence_detector (
    input clk, reset, in,
    output reg detected
);
    parameter S0=3'b000, S1=3'b001, S2=3'b010, S3=3'b011, S4=3'b100;
    reg [2:0] current_state, next_state;

    // State register (sequential)
    always @(posedge clk or posedge reset) begin
        if (reset) current_state <= S0;
        else current_state <= next_state;
    end

    // Next state logic (combinational)
    always @(*) begin
        case (current_state)
            S0: next_state = in ? S1 : S0;
            S1: next_state = in ? S1 : S2;
            S2: next_state = in ? S3 : S0;
            S3: next_state = in ? S4 : S2;
            S4: next_state = in ? S1 : S0;
            default: next_state = S0;
        endcase
    end

    // Output logic (Moore - depends only on current state)
    always @(*) begin
        detected = (current_state == S4) ? 1'b1 : 1'b0;
    end
endmodule

// Testbench
module tb_fsm_sequence_detector;
```

```

reg clk, reset, in;
wire detected;
fsm_sequence_detector uut (.clk(clk),.reset(reset),.in(in),.detected(detected));
always #5 clk = ~clk;
initial begin
clk=0; reset=1; in=0;
#10 reset=0;
// Apply sequence: 1 0 1 1
#10 in=1; #10 in=0; #10 in=1; #10 in=1;
#10 in=0; #10 in=1; #10 in=0; #10 in=1; #10 in=1;
#20 $finish;
end
initial $monitor("Time=%0t | in=%b | state=%b | detected=%b",
$time, in, uut.current_state, detected);
endmodule

```

2. D Flip-Flop (Positive Edge Triggered with Reset)

Implements a positive edge-triggered D flip-flop with synchronous reset. Produces Q and Q_bar outputs. Testbench applies all input combinations and verifies reset behaviour.

```

// D Flip-Flop (Positive Edge Triggered with Reset)
// Author: Thejaswini K | Maven Silicon Training

module d_flip_flop (
input clk, reset, d,
output reg q,
output wire q_bar
);
assign q_bar = ~q;

always @(posedge clk or posedge reset) begin
if (reset) q <= 1'b0;
else q <= d;
end
endmodule

// Testbench
module tb_d_flip_flop;
reg clk, reset, d;
wire q, q_bar;
d_flip_flop uut (.clk(clk),.reset(reset),.d(d),.q(q),.q_bar(q_bar));
always #5 clk = ~clk;
initial begin
clk=0; reset=1; d=0;

```

```

#10 reset=0;
#10 d=1; #10 d=0; #10 d=1; #10 d=1; #10 d=0;
#10 reset=1; #10 reset=0; #10 d=1;
#20 $finish;
end
initial $monitor("Time=%0t | reset=%b | d=%b | q=%b | q_bar=%b",
$time, reset, d, q, q_bar);
endmodule

```

3. Half Adder and Half Subtractor

Implements a Half Adder (Sum = A XOR B, Carry = A AND B) and Half Subtractor (Diff = A XOR B, Borrow = NOT A AND B) using dataflow modelling. Testbench displays complete truth tables for both circuits.

```

// Half Adder and Half Subtractor
// Author: Thejaswini K | Maven Silicon Training

module half_adder (
input a, b,
output sum, carry
);
assign sum = a ^ b; // XOR
assign carry = a & b; // AND
endmodule

module half_subtractor (
input a, b,
output diff, borrow
);
assign diff = a ^ b; // XOR
assign borrow = (~a) & b; // NOT a AND b
endmodule

// Testbench
module tb_half_adder_subtractor;
reg a, b;
wire sum, carry, diff, borrow;

half_adder ha (.a(a),.b(b),.sum(sum),.carry(carry));
half_subtractor hs (.a(a),.b(b),.diff(diff),.borrow(borrow));

initial begin
$display("=== Half Adder Truth Table ===");
$display("A B | Sum Carry");
a=0; b=0; #10; $display("%b %b | %b %b", a,b,sum,carry);

```

```

a=0; b=1; #10; $display("%b %b | %b %b", a,b,sum,carry);
a=1; b=0; #10; $display("%b %b | %b %b", a,b,sum,carry);
a=1; b=1; #10; $display("%b %b | %b %b", a,b,sum,carry);

$display("=== Half Subtractor Truth Table ===");
$display("A B | Diff Borrow");
a=0; b=0; #10; $display("%b %b | %b %b", a,b,diff,borrow);
a=0; b=1; #10; $display("%b %b | %b %b", a,b,diff,borrow);
a=1; b=0; #10; $display("%b %b | %b %b", a,b,diff,borrow);
a=1; b=1; #10; $display("%b %b | %b %b", a,b,diff,borrow);
$finish;
end
endmodule

```